

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- ... den Begriff der Berechenbarkeit und die Grenzen algorithmischer Lösbarkeit.
- ... die Bedeutung von Komplexitätsklassen wie P und NP.
- ... die Relevanz NP-vollständiger Probleme für die Informatik.

Sie können...

- ... erklären, warum RAM und Turing-Maschine gleich mächtig sind.
- ... berechenbare von nicht berechenbaren Problemen unterscheiden.
- ... das Halteproblem als Beispiel eines unentscheidbaren Problems erläutern.

Berechnungsmodelle

Berechnungsmodelle

Random Access Machine (RAM)



Bei den Komplexitätsbetrachtungen wird ein einfaches, aber utopisches Rechnermodell zugrunde gelegt: die ***Random Access Machine (RAM)***:

- Unendlich viele Speicherzellen, die in konstanter Zeit gelesen oder beschrieben werden
- Überschaubarer Befehlssatz (arithm. Operationen, Kontrollstrukturen, Speicherzugriff)

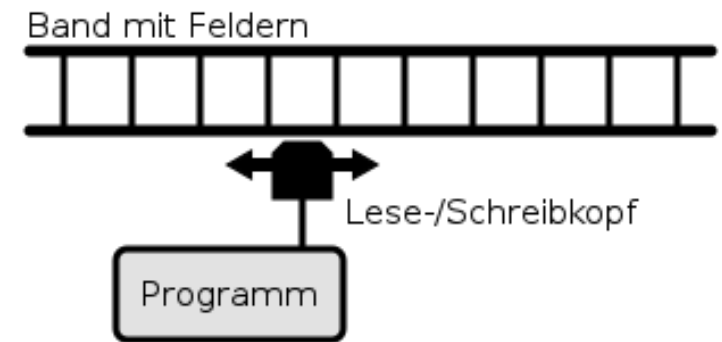
Berechnungsmodelle

Deterministische Turing-Maschine (TM)

Eine Turingmaschine ist ein **alternatives Berechnungsmodell**.

Sie besteht aus:

- Einem unendlich langen Band, auf dem Daten gespeichert werden können.
- Einem Schreib-Lese-Kopf, der sich in beiden Richtungen über das Band bewegen kann und jeweils ein Zeichen lesen und schreiben kann.
- Einem endl. Zustandsautomaten, dessen Übergänge vom aktuell gelesenen Zeichen determiniert sind („Programm“). Bei einem Übergang kann ein Zeichen geschrieben und der Schreib-Lese-Kopf um eine Position nach links oder rechts bewegt werden.



Der Input ist die initiale Beschriftung des Bands, der Output ist die Beschriftung des Bands, wenn ein Endzustand erreicht wird.

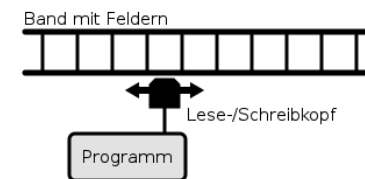
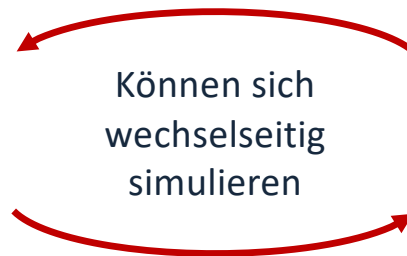
Berechnungsmodelle

Gleichwertigkeit von RAM und TM

Es wurde gezeigt, dass RAM und TM gleich mächtig sind, d.h. alles, was mit einer RAM berechnet werden kann, kann auch von einer TM berechnet werden und umgekehrt.



RAM



TM

Berechnungsmodelle

Turing Complete

Ein Berechnungsmodell, das genauso mächtig ist wie eine TM, wird als ***Turing-vollständiges System*** oder auch als ***Turing Complete*** bezeichnet.

Eine RAM ist also ***Turing Complete***.

Weitere Beispiele:

- TeX und damit auch LaTeX sind turing-vollständig. Das hat schon der Erfinder Donald Knuth bewiesen.
- Das "Game of Life" von John Conway ist turing-vollständig. Als Input dient die Startpopulation.
- Das Spiel „Minecraft“ ist turing-vollständig.

Berechenbarkeit

Berechenbarkeit

Grenzen

Gibt es Probleme, die sich durch die gezeigten Berechnungsmodelle, die ja über endliche Computer hinausgehen, nicht berechnet werden können?

Ja! Denn es gibt schon mehr Funktionen $f: \mathbb{Z} \rightarrow \mathbb{Z}$ als es Programme für eine Turing Maschine geben kann. D.h. es gibt Funktionen, für die es kein Programm gibt.

Beweis:

Wir zeigen: es gibt **überabzählbar unendlich** viele Funktionen $f: \mathbb{Z} \rightarrow \mathbb{Z}$,
aber nur **abzählbar unendlich** viele TM-Programme

Berechenbarkeit

Grenzen - Beweis

Es gibt überabzählbar unendlich viele Funktionen $f: \mathbb{Z} \rightarrow \mathbb{Z}$

Widerspruchsbeweis:

Sollte es nur abzählbar unendlich viele solche Funktionen geben, dann könnte man deren Wertetabellen in eine unendliche Liste untereinander schreiben.

	1	2	3	4	5
f_1	4+1	2	3	7	...
f_2	1	9+1	3	1	...
f_3	7	3	3+1	6	...
f_4	6	2	7	1+1	...
f_5	2	8	5	5	...
...					

Dann ist aber die Funktion, die sich durch Addition von 1 zu jedem Element der Diagonale ergibt, nicht enthalten...

Berechenbarkeit

Grenzen - Beweis

Es gibt nur abzählbar unendlich viele TM-Programme

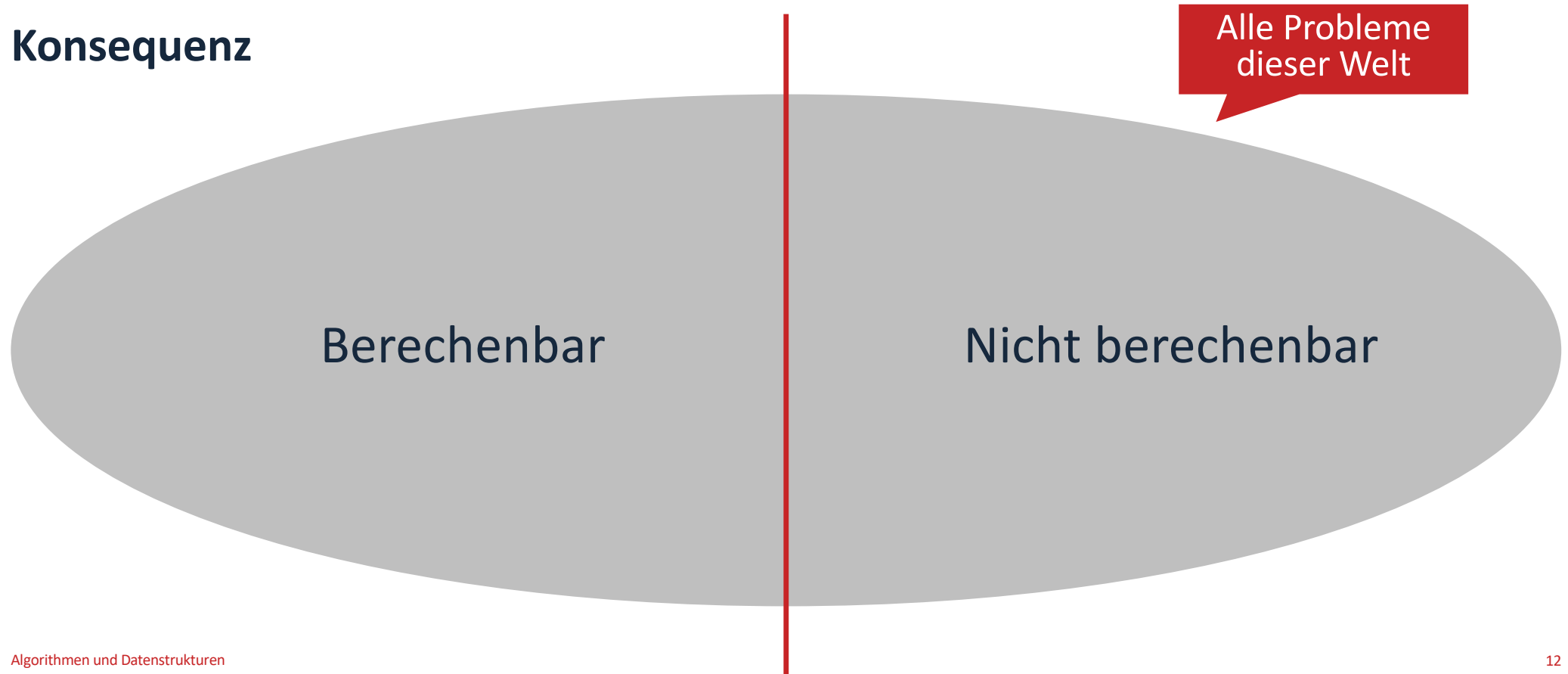
Beweis:

Betrachtet wird das gleich mächtige RAM-Berechnungsmodell. Alle Programme bestehen aus einem endlichen Satz von Befehlen, die man zwar zu unendlich vielen Programmen zusammensetzen kann. Diese können aber problemlos in eine unendliche Liste überführt werden, indem man alle Programme als String betrachtet und alphabetisch sortiert.

→ Die Anzahl der Programme ist abzählbar unendlich.

Berechenbarkeit

Konsequenz



Berechenbarkeit

Beispiel für eine nicht berechenbare Funktion

Behauptung

Sei X ein Programm und y ein Input, dann ist die Funktion

$$H(X, y) = \begin{cases} 0 & \text{falls } X(y) \text{ terminiert} \\ 1 & \text{sonst} \end{cases}$$

nicht berechenbar.

Berechenbarkeit

Beispiel für eine nicht berechenbare Funktion

Widerspruchsbeweis

Wenn so eine Funktion $H(X, y)$ berechnet werden könnte, dann könnte man sie in folgenden Programm Z verwenden:

 $Z(y)$

1: **repeat**

2: **until** $H(Z, y) = 1$

Falls Z beim Input y terminiert, liefert $H(X, y)$ den Wert 0 und die Schleife wird wiederholt, d.h. Z terminiert nicht.

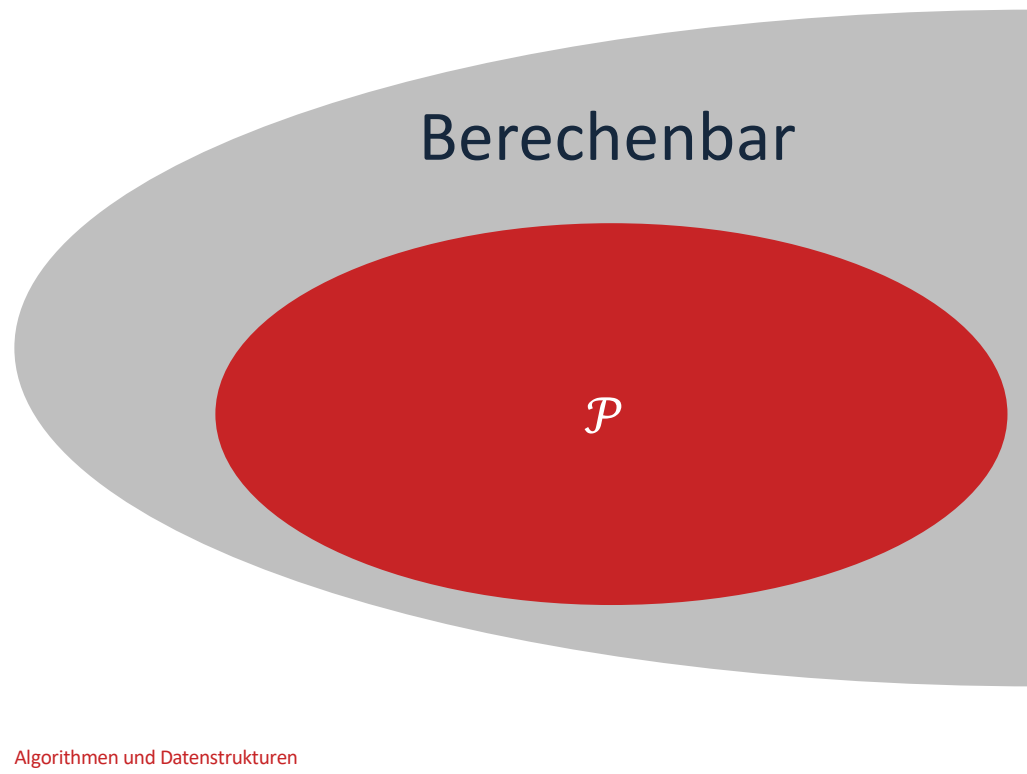
Falls Z aber beim Input y nicht terminiert, liefert $H(X, y)$ den Wert 1 und das Programm endet.

→ Das Halteproblem ist nicht entscheidbar,
 H nicht berechenbar

Komplexitätsklassen

Komplexitätsklassen

Effizient lösbare Probleme



Gegenstand der Vorlesung waren Fragestellungen, für die es Algorithmen mit der Zeitkomplexität

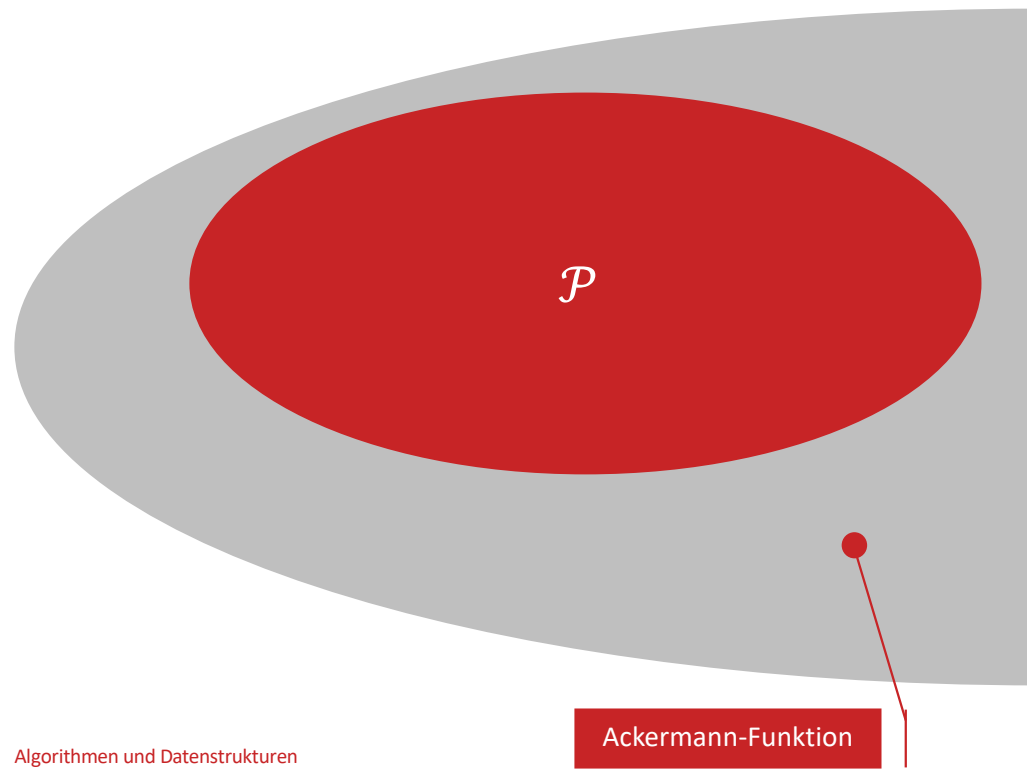
$$O(n^i) \text{ mit } i \in \mathbb{N}_0$$

gibt.

Diese Fragestellungen gehören zu $\mathcal{P}$, der Klasse der in polynomialer Zeit von einer TM lösbaren Probleme. Solche Fragestellungen werden auch als effizient lösbar angesehen.

Komplexitätsklassen

Ackermann-Funktion



Definition:

$$\begin{aligned} A(0, n) &= n + 1 \\ A(m, 0) &= A(m - 1, 1) \\ A(m, n) &= A(m - 1, A(m, n - 1)) \end{aligned}$$

Selbst für kleine Werte von m und n kann die Berechnung von $A(m, n)$ eine erhebliche Anzahl von Rechenschritten erfordern, die exponentiell in Bezug auf m und n wachsen.

Komplexitätsklassen

Nicht-Deterministische Turing-Maschine (NTM)

Eine Nicht-Deterministische Turingmaschine ist eine Erweiterung der Turingmaschine. In einer NTM

- kann es für einen Zustand mehrere Übergänge bei gleichem gelesenen Zeichen geben
- werden alle diese Übergänge gleichberechtigt parallel ausgeführt
- ist es ausreichend, wenn einer der parallelen Pfade einen Endzustand erreicht



Die Grenze der Berechenbarkeit wird durch NTM nicht verschoben.

Aber: durch die Parallelität kann sich die Zeitkomplexität verändern: Es können u.U. Probleme mit NTM in polynomialer Zeit gelöst werden, die nicht in $\mathcal{P}$ waren.

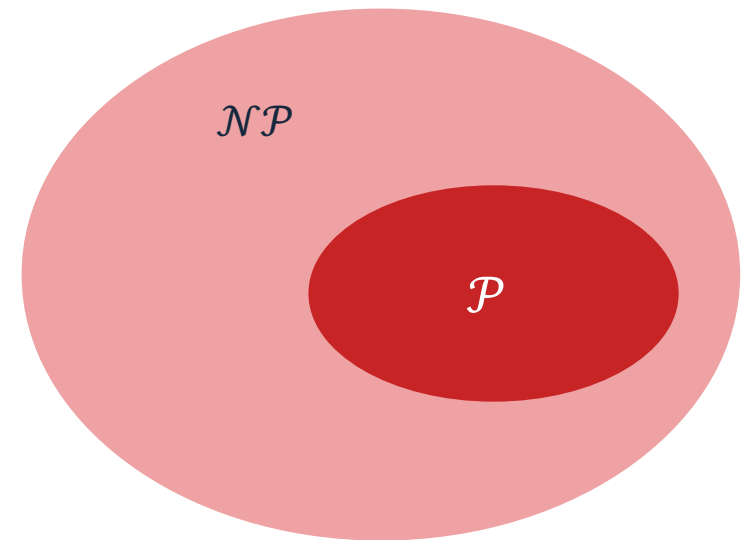
Komplexitätsklassen

P = NP ?

Alle von einer NTM in polynomialer Zeit lösbaren Probleme werden auch als $\mathcal{NP}$ bezeichnet.

Es ist klar, dass $\mathcal{P} \subseteq \mathcal{NP}$.

Erstaunlicherweise hat man aber noch nicht zeigen können, dass es Probleme gibt, die in $\mathcal{NP}$ enthalten sind, nicht aber in $\mathcal{P}$. Es könnte also $\mathcal{P} = \mathcal{NP}$ sein.



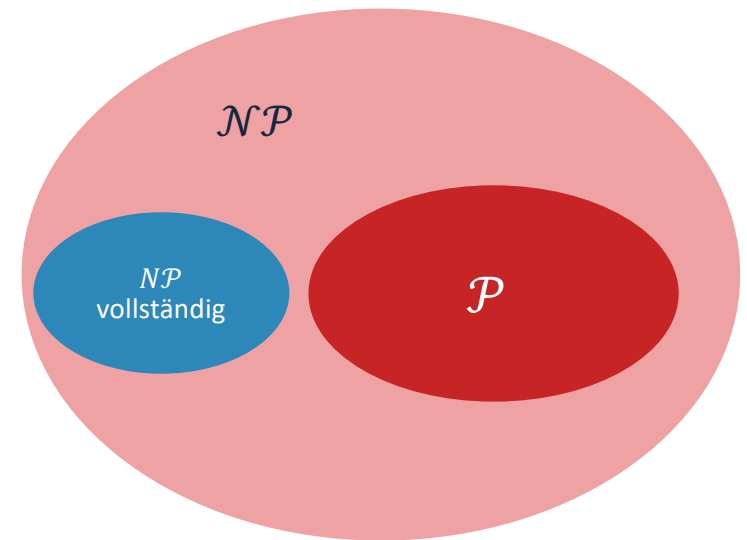
Komplexitätsklassen

NP-vollständige Probleme

Man kann zeigen, dass es in NP einige Probleme gibt, auf die sich alle anderen Probleme in NP in polynomialer Zeit reduzieren lassen.

Diese Probleme stellen sozusagen die schwierigsten Probleme in NP dar und heißen **NP vollständig**.

Wenn für eines dieser NP-vollständigen Probleme ein Algorithmus mit polynomialer Laufzeit gefunden werden könnte, wäre $NP=P$ bewiesen.



Komplexitätsklassen

NP-vollständige Probleme

Beispiele:

Traveling Salesman Problem

Was ist die kostengünstigste Rundreise durch n Städte?

Erfüllbarkeitsproblem

Gibt es eine Belegung der n Variablen einer booleschen Formel, die den Ausdruck wahr macht?

Komplexitätsklassen

Offene Frage

In der theoretischen Informatik ist bis heute unklar:

- Gilt $P = NP$?
- Und damit: existieren effiziente Algorithmen für NP-vollständige Probleme?

" $P = NP$?" ist eines der sechs noch ungelösten Millennium-Probleme und für seine Lösung ist ein Preis von 1 Mio. US-Dollar ausgesetzt."

Vielen Dank für die Aufmerksamkeit!